

System Programming

Memory Management

Dongwann Kang

Associate Professor
Department of Computer Science and Engineering

Dynamic Memory Allocation

- Dynamic memory allocation (aka. heap allocation)
 - A technique programming languages use for reserving memory when objects are created

- Java

```
String str = new String("abcde");
```

- C++

```
int size;  
cin >> size;  
int array[] = new int[size];
```

- Built-in runtime heap managers in C, C++, and Java
 - Heap managers handle programmatic requests for storage allocation and deallocation
 - They generally allocate a large block of memory from the OS when the program starts up
 - They create a *free list* of pointers to storage blocks
 - Allocating a storage
 - The heap manager looks for the first available block large enough from the *free list*, and marks the block as reserved
 - Then it returns a pointer to the block
 - Deallocating the storage
 - The heap frees up the block, returning it to the free list

Dynamic Memory Allocation

- Dynamic memory allocation in assembly language programs
 - You can request multiple blocks of memory of varying sizes from Windows, using several Win32 API functions

Function	Description
GetProcessHeap	Returns a 32-bit integer handle to the program's existing heap area in EAX. If the function succeeds, it returns a handle to the heap in EAX. If it fails, the return value in EAX is NULL.
HeapAlloc	Allocates a block of memory from a heap. If it succeeds, the return value in EAX contains the address of the memory block. If it fails, the returned value in EAX is NULL.
HeapCreate	Creates a new heap and makes it available to the calling program. If the function succeeds, it returns a handle to the newly created heap in EAX. If it fails, the return value in EAX is NULL.
HeapDestroy	Destroys the specified heap object and invalidates its handle. If the function succeeds, the return value in EAX is nonzero.
HeapFree	Frees a block of memory previously allocated from a heap, identified by its address and heap handle. If the block is freed successfully, the return value is nonzero.
HeapReAlloc	Reallocates and resizes a block of memory from a heap. If the function succeeds, the return value is a pointer to the reallocated memory block. If the function fails and you have not specified HEAP_GENERATE_EXCEPTIONS, the return value is NULL.
HeapSize	Returns the size of a memory block previously allocated by a call to HeapAlloc or HeapReAlloc. If the function succeeds, EAX contains the size of the allocated memory block, in bytes. If the function fails, the return value is SIZE_T - 1. (SIZE_T equals the maximum number of bytes to which a pointer can point.)

Dynamic Memory Allocation

- Dynamic memory allocation in assembly language programs
 - *GetProcessHeap*
 - Returns a handle to the program's existing heap in EAX
 - *HeapCreate*
 - Creates a new private heap for the current program, and returns the handle to the heap in EAX

```
HeapCreate PROTO,  
    flOptions:DWORD,          ; heap allocation options  
    dwInitialSize:DWORD,      ; initial heap size, in bytes  
    dwMaximumSize:DWORD       ; maximum heap size, in bytes
```

- Default *flOptions* is NULL
- The initial and maximum heap sizes are rounded up to the next page boundary
- When calls to *HeapAlloc* exceed the initial heap size, it will grow

- *HeapDestroy*
 - Destroys an existing private heap (created by *HeapCreate*)

```
HeapDestroy PROTO,  
    hHeap:DWORD               ; heap handle
```

Dynamic Memory Allocation

- Dynamic memory allocation in assembly language programs

- *HeapAlloc*

- Allocates a memory block from an existing heap, and returns a pointer to the new storage in EAX

```
HeapAlloc PROTO,  
    hHeap:HANDLE,          ; handle to existing heap block  
    dwFlags:DWORD,        ; heap allocation control flags  
    dwBytes:DWORD          ; number of bytes to allocate
```

- You can optionally set *dwFlags* to HEAP_ZERO_MEMORY, which sets the memory block to all zeros

- *HeapFree*

- Frees a block of memory previously allocated from a heap

```
HeapFree PROTO,  
    hHeap:HANDLE,  
    dwFlags:DWORD,  
    lpMem:DWORD
```

- Default *dwFlags* is NULL

- If you encounter an error when calling *HeapCreate*, *HeapDestroy*, or *GetProcessHeap*, you can get details by calling the *GetLastError* API function

Dynamic Memory Allocation

- HeapTest Programs
 - An example that creates and fills a 1000-byte array

<pre> INCLUDE Irvine32.inc .data ARRAY_SIZE = 1000 FILL_VAL EQU 0FFh hHeap HANDLE ? ; handle to the process heap pArray DWORD ? ; pointer to block of memory newHeap DWORD ? ; handle to new heap str1 BYTE "Heap size is: ",0 .code main PROC INVOKE GetProcessHeap ; get handle prog's heap .IF eax == NULL ; if failed, display message call WriteWindowsMsg jmp quit .ELSE mov hHeap,eax ; success .ENDIF call allocate_array jnc arrayOk ; failed (CF = 1)? call WriteWindowsMsg call Crlf jmp quit arrayOk: ; ok to fill the array call fill_array call display_array call Crlf ; free the array INVOKE HeapFree, hHeap, 0, pArray quit: exit main ENDP </pre>	<pre> ;----- allocate_array PROC USES eax ; ; Dynamically allocates space for the array. ; Receives: EAX = handle to the program heap ; Returns: CF = 0 if the memory allocation succeeds. ;----- INVOKE HeapAlloc, hHeap, HEAP_ZERO_MEMORY, ARRAY_SIZE .IF eax == NULL stc ; return with CF = 1 .ELSE mov pArray,eax ; save the pointer clc ; return with CF = 0 .ENDIF ret allocate_array ENDP ;----- fill_array PROC USES ecx edx esi ; ; Fills all array positions with a single character. ; Receives: nothing ; Returns: nothing ;----- mov ecx,ARRAY_SIZE ; loop counter mov esi,pArray ; point to the array L1: mov BYTE PTR [esi],FILL_VAL ; fill each byte inc esi ; next location loop L1 ret fill_array ENDP </pre>	<pre> ;----- display_array PROC USES eax ebx ecx esi ; ; Displays the array ; Receives: nothing ; Returns: nothing ;----- mov ecx,ARRAY_SIZE ; loop counter mov esi,pArray ; point to the array L1: mov al,[esi] ; get a byte mov ebx,TYPE BYTE call WriteHexB ; display it inc esi ; next location loop L1 ret display_array ENDP END main </pre>
--	--	---

Dynamic Memory Allocation

- HeapTest Programs
 - An example that repeatedly allocate large blocks of memory until the heap size is exceeded

```
INCLUDE Irvine32.inc

.data
HEAP_START = 2000000      ; 2 MByte
HEAP_MAX   = 400000000    ; 400 MByte
BLOCK_SIZE = 500000       ; .5 MByte

hHeap HANDLE ?           ; handle to the heap
pData DWORD ?            ; pointer to block

str1 BYTE 0dh,0ah,"Memory allocation failed",0dh,0ah,0

.code
main PROC
    INVOKE HeapCreate, 0,HEAP_START, HEAP_MAX
    .IF eax == NULL      ; failed?
        call WriteWindowsMsg
        call Crlf
        jmp quit
    .ELSE
        mov hHeap,eax    ; success
    .ENDIF

    mov ecx,2000         ; loop counter
L1: call allocate_block   ; allocate a block
    .IF Carry?           ; failed?
        mov edx,OFFSET str1 ; display message
        call WriteString
        jmp quit
    .ELSE
        ; no: print a dot to
        mov al, '.'      ; show progress
        call WriteChar
    .ENDIF

    ;call free_block      ; enable/disable this line
    loop L1
```

```
quit:
    INVOKE HeapDestroy, hHeap ; destroy the heap
    .IF eax == NULL          ; failed?
        call WriteWindowsMsg ; yes: error message
        call Crlf
    .ENDIF

    exit
main ENDP

allocate_block PROC USES ecx
    ; allocate a block and fill with all zeros.
    INVOKE HeapAlloc, hHeap, HEAP_ZERO_MEMORY, BLOCK_SIZE

    .IF eax == NULL
        stc                                ; return with CF = 1
    .ELSE
        mov pData,eax                     ; save the pointer
        clc                                ; return with CF = 0
    .ENDIF
    ret
allocate_block ENDP

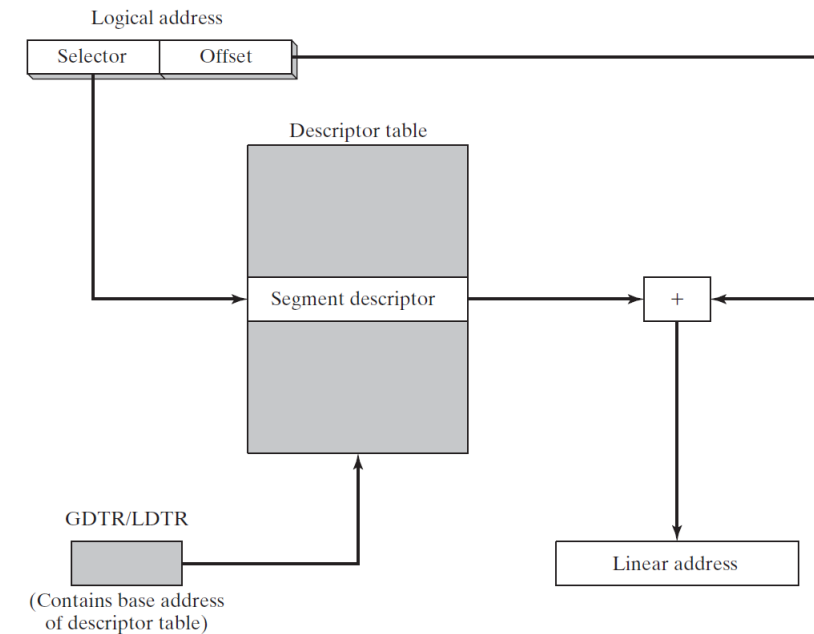
free_block PROC USES ecx
    INVOKE HeapFree, hHeap, 0, pData
    ret
free_block ENDP
END main
```

x86 Memory Management

- Some of the x86 memory-management terms
 - Multitasking
 - It permits multiple programs to run at the same time
 - The processor divides its time among all of the running programs
 - Segments
 - Variable-sized areas of memory used by a program containing either code or data
 - Segmentation
 - It provides a way to isolate memory segments from each other
 - This permits multiple programs to run simultaneously without interfering with each other
 - Segment descriptor
 - It identifies and describes a single memory segment
 - It contains information about the segment's base address, access rights, size limit, type, and usage
 - Segment selector
 - A 16-bit value stored in a segment register (CS, DS, SS, ES, FS, or GS)
 - Logical address
 - A combination of a segment selector and a 32-bit offset

x86 Memory Management

- 1. Linear Addresses
 - Translating Logical Addresses to Linear Addresses
 - On a multitasking OS, each program has its own unique area for data
 - Suppose three programs each had a variable at offset 200h
 - x86 processors use a one- or two-step process to convert each variable's offset into a unique address
 - The first step combines a segment value with a variable's offset to create a linear address
 - » This linear address could be the variable's physical address
 - » But OSs such as MS-Windows and Linux employ *paging*
 - The second step (called *page translation*) converts a linear address to a physical address
 - Calculating linear address
 - Each segment selector points to a segment descriptor (in a descriptor table)
 - » A segment descriptor contains the base address of a segment
 - 32-bit linear address = the segment's base address + 32-bit offset

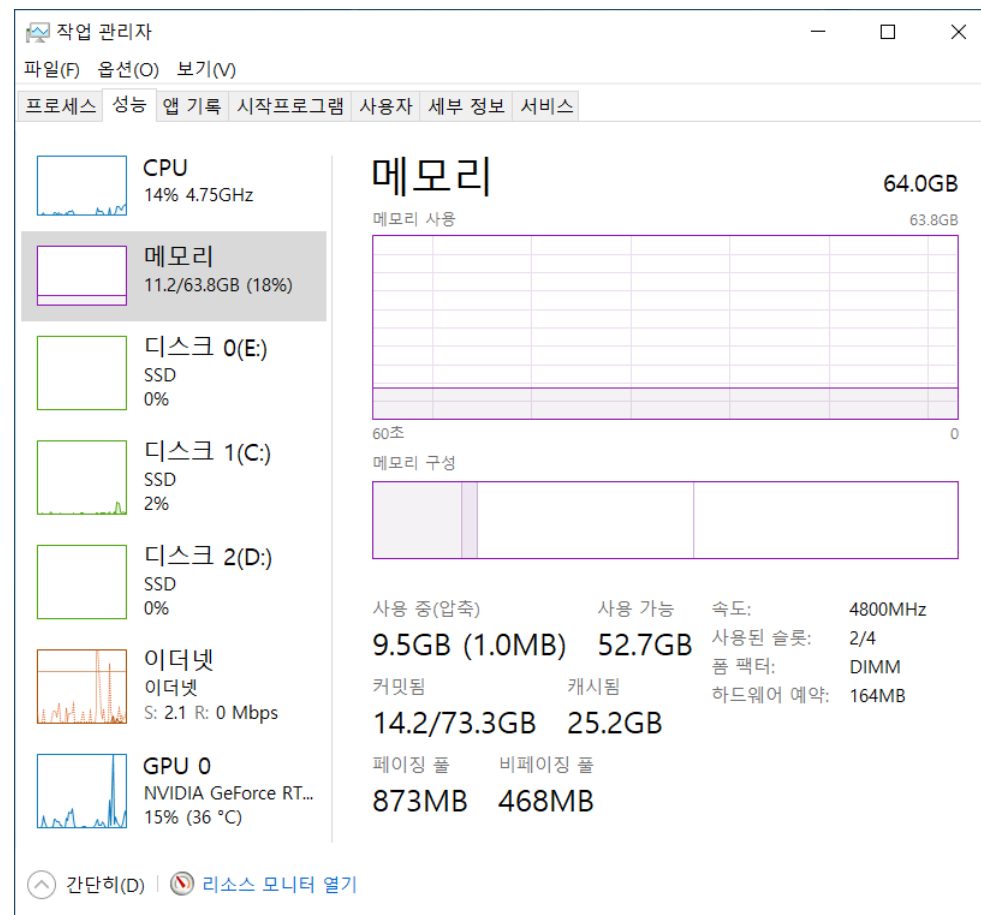


x86 Memory Management

- 1. Linear Addresses
 - Paging
 - By using paging, a computer can run a combination of programs that would not otherwise fit into memory
 - The processor initially loads only part of a program in memory while keeping the remaining parts on disk
 - The memory used by the program is divided into small units called *pages* (typically 4 Kbyte each)
 - As each program runs, the processor selectively unloads inactive pages from memory and loads other pages that are immediately required
 - The OS maintains a *page directory* and a set of *page tables* to keep track of the pages used by all programs currently in memory
 - *Page translation*
 - The conversion from the linear address into a physical address
 - If the requested page is not currently in memory, the processor interrupts the program and issues a *page fault*
 - The OS copies the required page from disk into memory before the program can resume
 - From the point of view of an application program, page faults and page translation happen automatically

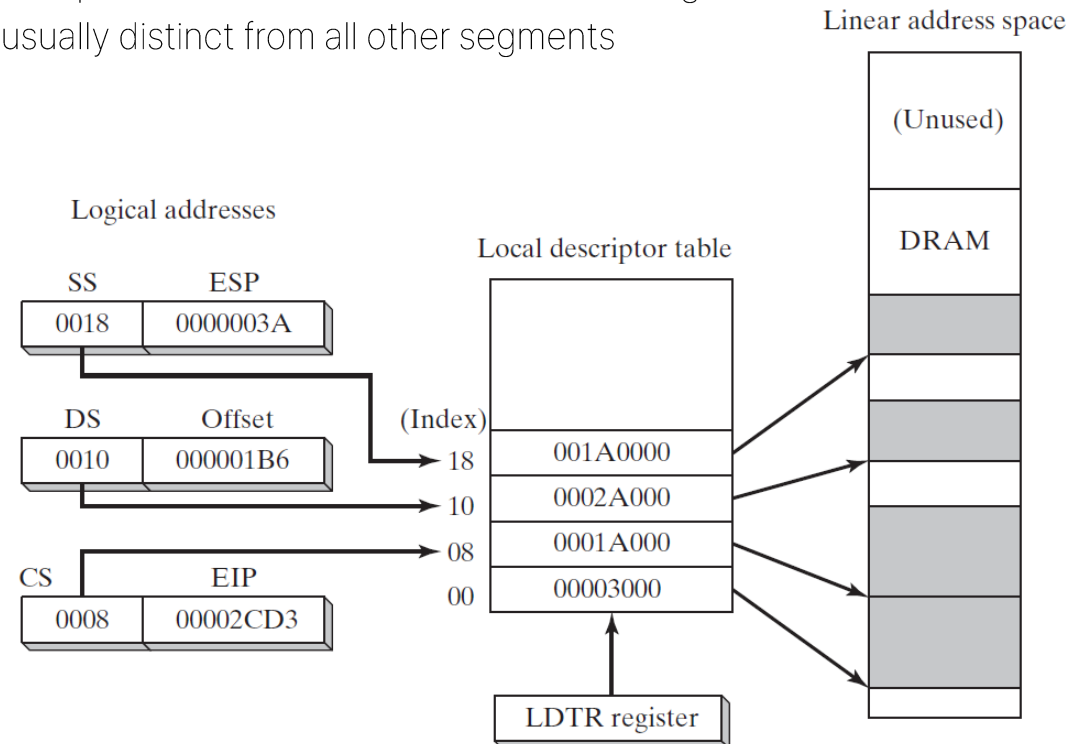
x86 Memory Management

- 1. Linear Addresses
 - Paging
 - By using paging, a computer can run a combination of programs that would not otherwise fit into memory



x86 Memory Management

- 1. Linear Addresses
 - Descriptor Tables
 - Global Descriptor Table (GDT)
 - A single GDT is created when the OS switches the processor into protected mode during boot up
 - Local Descriptor Tables (LDT)
 - In a multitasking OS, each task or program is usually assigned its own table of segment descriptors, called an LDT
 - The LDTR register contains the address of the program's LDT
 - Each segment descriptor contains the base address of a segment within the linear address space
 - This segment is usually distinct from all other segments



x86 Memory Management

- 1. Linear Addresses
 - Segment Descriptor Details
 - Base address
 - A 32-bit integer that defines the starting location of the segment in the 4 Gbyte linear address space
 - Privilege level
 - Each segment can be assigned a privilege level between 0 and 3, where 0 is the most privileged, usually for OS kernel code
 - Segment type
 - The type of segment, type of access, and the direction the segment can grow
 - » Data (including Stack) segments can be read-only or read/write and can grow either up or down
 - » Code segments can be execute-only or execute/read-only
 - Segment present flag
 - This bit indicates whether the segment is currently present in physical memory
 - Granularity flag
 - Determines the interpretation of the Segment limit field (0: byte units, 1: 4096-byte units)
 - Segment limit
 - This 20-bit integer specifies the size of the segment. It depends on the Granularity flag
 - » The segment size ranges from 1 Byte to 1 MByte (when the Granularity flag is clear)
 - » The segment size ranges from 4 KByte to 4 Gbyte (when the Granularity flag is set)

x86 Memory Management

- 2. Page Translation
 - The processor translates a 32-bit linear address into a 32-bit physical address
 - There are three structures used in the process:
 - Page directory: An array of up to 1024 32-bit page-directory entries
 - Page table: An array of up to 1024 32-bit page-table entries
 - Page: A 4 KByte or 4 MByte address space
 - A linear address is divided into three fields:
 - A pointer to a page-directory entry
 - A pointer to a page-table entry
 - An offset into a page frame
 - Control register (CR3) contains the starting address of the page directory

x86 Memory Management

- 2. Page Translation

- Page translation process:

1. The linear address references a location in the linear address space
2. The 10-bit directory field in the linear address is an index to a page-directory entry
 - The page directory entry contains the base address of a page table
3. The 10-bit table field in the linear address is an index into the page table identified by the page-directory entry
 - The page-table entry at that position contains the base location of a page in physical memory.
4. The 12-bit offset field in the linear address is added to the base address of the page, generating the exact physical address of the operand

